

Causey software engineering

How to commit, push, and land work on **dev**

Causey

September 5, 2026

For: Divine **Repo:** Causey

Integration branch: **dev** **Production:** **main** — do not open PRs here unless asked

Adam and Divine both work in this repo at the same time. Adam lands most product work and production fires directly on **dev**. Divine does not share **dev** as a working copy. Divine uses a short-lived branch, keeps it current with **dev**, and opens a pull request **into dev**. That is how both people ship in parallel without overwriting each other.

1 The rule

Who	What they push
Adam	Commits and pushes to dev for fires and most product work.
Divine	A feature branch such as ui/claim-invite . Pull request into dev .
Nobody	Force-push dev . Delete dev . Push to main .

If **dev** moved three times while Divine was coding, that is expected. Rebase the branch onto latest **origin/dev**, then open or update the PR. Never push an old local **dev** to GitHub.

Never do these

- `git push` to **dev** or **main**.
- `git push --force` or `--force-with-lease` on **dev** or **main**.
- Commit on local **dev**. Create a branch first.
- Work for days without updating from **origin/dev**.
- Open a pull request against **main**.
- Mix unrelated UI, copy, and backend changes in one PR. One job, one branch.

Force-with-lease is allowed **only on Divine's feature branch** after a rebase. It is never allowed on **dev**.

2 Every ticket, in order

```
git fetch origin
git checkout -b ui/short-name origin/dev
# ... edit, then:
git add -p
git commit
# before you ask for review, and again if the PR sits overnight:
git fetch origin
git rebase origin/dev
git push -u origin HEAD
# after a rebase of an already-pushed branch:
git push --force-with-lease
```

Then open a GitHub pull request with **base dev** and **compare = your branch**. Do not merge it unless Adam says to.

3 Start a task

Always branch from the remote, not from a stale local **dev**. Name the branch so the job is obvious.

Prefix	Use for
ui/...	Layout, copy, styling, empty states, forms, responsive work.
fix/...	A specific bug. Name the screen or symptom.
chore/...	Small cleanup that is not a user-facing change.

Examples: `ui/claim-invite-auth`, `fix/search-empty-state`, `ui/org-people-spacing`.

Before you start, say which screen you are on. If Adam is already in those files, pick a different ticket. Same files at the same time is how merges get ugly even when git is used correctly.

4 Commits

Commit on your branch only. Keep commits small enough that a reviewer can see why each one exists. Messages in this repo are why-focused and imperative: what the change does for the user, not a file list.

```
# good
Fix the claim page so a signed-out invitee can sign in and continue

# bad
update stuff / fix / ui changes / wip
```

Do not commit `.env`, credentials, or secrets. Do not add listings, fees, partner names, or counts that are not real product data.

5 Stay current with dev

Adam will push to `dev` while Divine is still working. Update before review, and again if the PR sits overnight.

```
git fetch origin
git rebase origin/dev
```

If git stops on a conflict:

1. Open the listed files. Keep Adam's firefixes unless your change is supposed to replace that exact line.
2. `git add` the resolved files, then `git rebase --continue`. Repeat until it finishes.

To abort: `git rebase --abort`. The branch returns to how it was before the rebase.

After a successful rebase of a branch that was already on GitHub, update it with force-with-lease (Divine's branch only):

```
git push --force-with-lease
```

If that is rejected, someone else pushed to the branch — stop and ask. Do not switch to `--force`.

If GitHub says the branch is out of date, that means `dev` moved. It does not mean Divine should push to `dev`. Rebase onto `origin/dev`, force-with-lease the feature branch, refresh the PR. If it is still blocked, paste the conflict files in Slack and wait. Do not resolve by taking “theirs” or “ours” blindly.

6 Pull request

First push of a new branch: `git push -u origin HEAD`. Later pushes with no rebase: `git push`.

- **Base:** `dev` **Compare:** your branch
- **Title:** same style as a good commit message
- **Body:** what changed, which screen to click, and any risk (shared components, CSS, auth)

Adam reviews and merges so a fire in the same files does not land on top of an unseen UI change. Prefer squash merge when it is offered. One PR becomes one commit on `dev`.

After merge, delete the remote branch (GitHub can do this on merge). Do not keep using the old branch. Do not merge `dev` into a finished branch “to be safe.” Start the next task from latest `origin/dev`:

```
git fetch origin
git checkout -b ui/next-task origin/dev
```

A good PR

- Branched from current `origin/dev`, then rebased onto `origin/dev` before review.
- One user-visible job.
- Reuses existing classes and tokens (`.field`, `.cta-enabled`, `.section-rule`, `.card-lift`). No one-off hex colors or `text-[13px]`.
- Copy is specific and honest. No “Beta,” “Live,” or “Coming soon” badges. No fabricated listings, fees, or counts.
- External links open in a new tab with an `aria-label`. CTAs name the next action (“Search tournaments”), not “Learn more” or “Get started.”

7 Lanes

Git cannot save anyone if Adam and Divine rewrite the same component on the same day.

- Say what screen you are on (claim invite, org people, search empty state).
- If Adam is already in those files, pick a different ticket.
- Keep the PR to one screen or one bug. A three-day branch that touches `globals.css` plus five pages will collide.

Shared files that need extra care: `app/globals.css`, layout chrome, auth components, invitation and claim paths, and anything under `lib/` that multiple screens import. If you need a one-line token change in `globals.css`, make that PR tiny and land it first.

8 Emergency: committed on dev by mistake

Do not push. Do not reset other people’s history. If the commit exists only on Divine’s machine:

```
git fetch origin
git branch ui/rescue-name
git reset --hard origin/dev
git checkout ui/rescue-name
```

The commit is now on `ui/rescue-name`. Rebase onto `origin/dev` if needed, push the branch, open a PR.

If the commit was already pushed to `dev`, stop and tell Adam. Do not force-push to undo it.

9 Desk card

Print this page. Tape it next to the laptop.

Every ticket

```
git fetch origin
git checkout -b ui/short-name origin/dev
git add -p && git commit
git fetch origin && git rebase origin/dev
git push -u origin HEAD
# PR base = dev. Do not merge unless Adam says to.
# After rebase of a pushed branch:
git push --force-with-lease
```

Commands

You want to	Run
Start work	git fetch origin then git checkout -b ui/name origin/dev
Save work	git add -p then git commit
Catch up to Adam	git fetch origin then git rebase origin/dev
Publish a new branch	git push -u origin HEAD
Update the PR after rebase	git push --force-with-lease
See whether you are behind	git fetch origin then git status

Never: push dev or main; force-push dev; commit on local dev; open a PR to main; work days without rebasing onto origin/dev.

The working agreement in one sentence: **Adam lands on dev; Divine lands through a short branch and a PR into dev; rebase often; never force-push dev; do not edit the same files on the same day without saying so.**